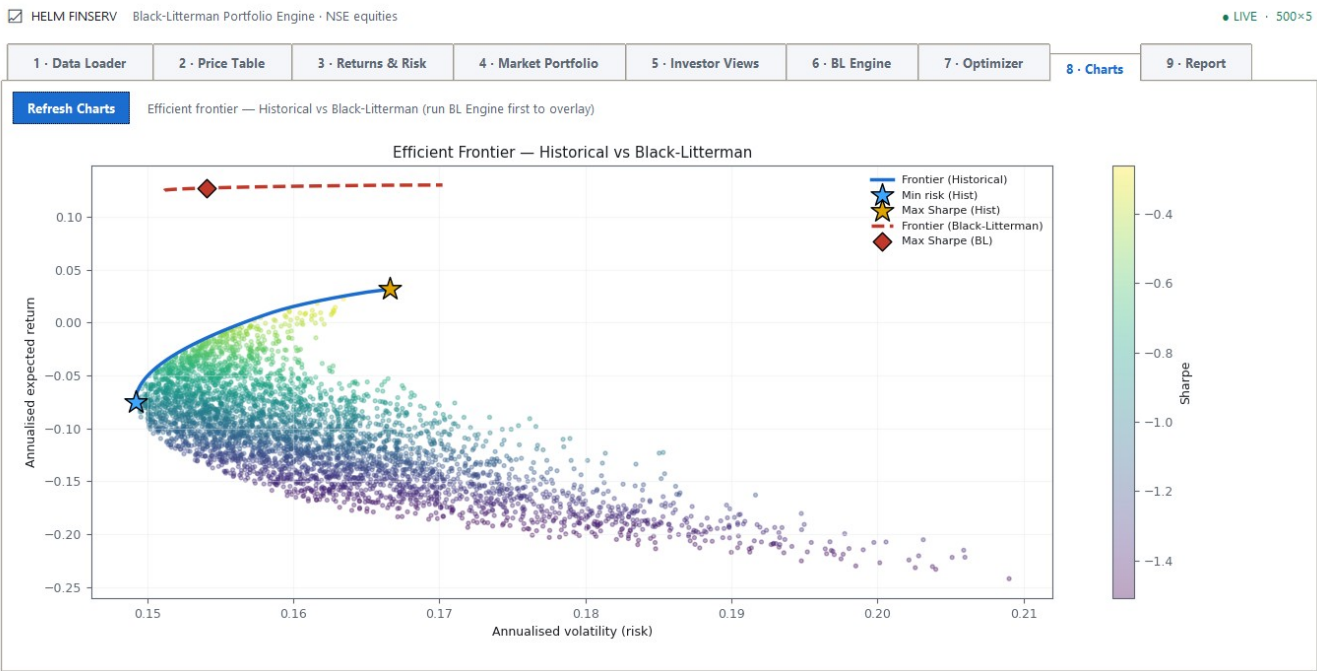


Black Litterman Portfolio Optimisation Engine

A confidence-weighted framework for converting market equilibrium and investor views into constrained portfolio allocations



Historical and Black Litterman efficient frontiers under consistent portfolio constraints

ASHISH KUMAR A

Quantitative Finance Research | Python | Portfolio Construction

What this project solves

Traditional mean variance optimisation can react aggressively to noisy historical averages. This project creates a more disciplined allocation process: it starts from market-cap-weighted equilibrium expectations, blends in explicit investment views according to confidence, and then optimises the resulting return and risk estimates under practical weight limits.

End to end research pipeline

Stage	Question answered	Output
Price history	What happened?	Aligned closing prices
Returns and risk	How did assets behave?	Return, volatility and covariance
Market equilibrium	What is already priced in?	Implied equilibrium returns
Investor views	What do I believe?	Absolute and relative views
Bayesian update	How strongly should views matter?	Posterior expected returns
Optimisation	How should capital be allocated?	Constrained portfolio weights
Evaluation	Did the portfolio improve?	Risk, Sharpe, concentration and turnover

Core achievement

The engine converts subjective forecasts into a mathematically controlled portfolio decision. Views influence the allocation without completely replacing the market prior, reducing the instability associated with raw historical-return optimisation.

- Standalone Python research engine with reproducible demonstration data and CSV input support.
- Consistent treatment of total returns, excess returns, covariance and annualisation.
- Long-only optimisation with configurable minimum and maximum position weights.
- Direct comparison of historical Markowitz and Black Litterman portfolios.

Price history and data preparation

The first stage aligns closing prices across the selected securities. Clean, synchronized observations are essential because every later return and covariance estimate depends on the same dates.

HELM FINSERV Black-Litterman Portfolio Engine · NSE equities ● LIVE · 500×5

1 · Data Loader	2 · Price Table	3 · Returns & Risk	4 · Market Portfolio	5 · Investor Views	6 · BL Engine	7 · Optimizer	8 · Charts	9 · Report
-----------------	-----------------	--------------------	----------------------	--------------------	---------------	---------------	------------	------------

Closing prices (most recent sessions)

Date	RELIANCE	TCS	HDFCBANK	INFY	SBIN
2026-03-09	1424.00	2527.40	840.70	1315.00	1098.50
2026-03-06	1404.80	2557.60	857.05	1308.40	1143.00
2026-03-05	1389.40	2578.80	877.75	1305.80	1169.50
2026-03-04	1345.00	2587.80	868.65	1307.40	1174.50
2026-03-02	1358.00	2613.50	879.40	1288.90	1189.90
2026-02-27	1393.90	2637.40	887.75	1300.10	1201.70
2026-02-26	1406.80	2647.70	898.60	1289.10	1209.50
2026-02-25	1398.50	2629.30	907.60	1290.10	1200.10
2026-02-24	1428.80	2573.70	910.50	1275.50	1223.30
2026-02-23	1428.00	2676.30	923.60	1327.50	1227.80
2026-02-20	1419.40	2686.20	911.85	1353.20	1216.10
2026-02-19	1409.50	2677.90	915.60	1370.50	1205.70
2026-02-18	1441.30	2694.90	924.70	1373.70	1218.90
2026-02-17	1423.00	2717.40	925.70	1391.20	1213.40
2026-02-16	1437.10	2706.60	925.80	1365.60	1208.10
2026-02-13	1419.60	2692.20	903.90	1369.10	1198.60
2026-02-12	1448.90	2750.10	920.60	1386.00	1192.40
2026-02-11	1468.70	2909.80	927.10	1471.90	1182.90
2026-02-10	1458.50	2984.60	932.40	1497.80	1144.10
2026-02-09	1461.60	2948.20	937.25	1497.20	1146.00
2026-02-06	1450.80	2941.60	941.10	1507.10	1066.40

Wednesday, September 16, 2026

Aligned closing-price observations used by the research pipeline

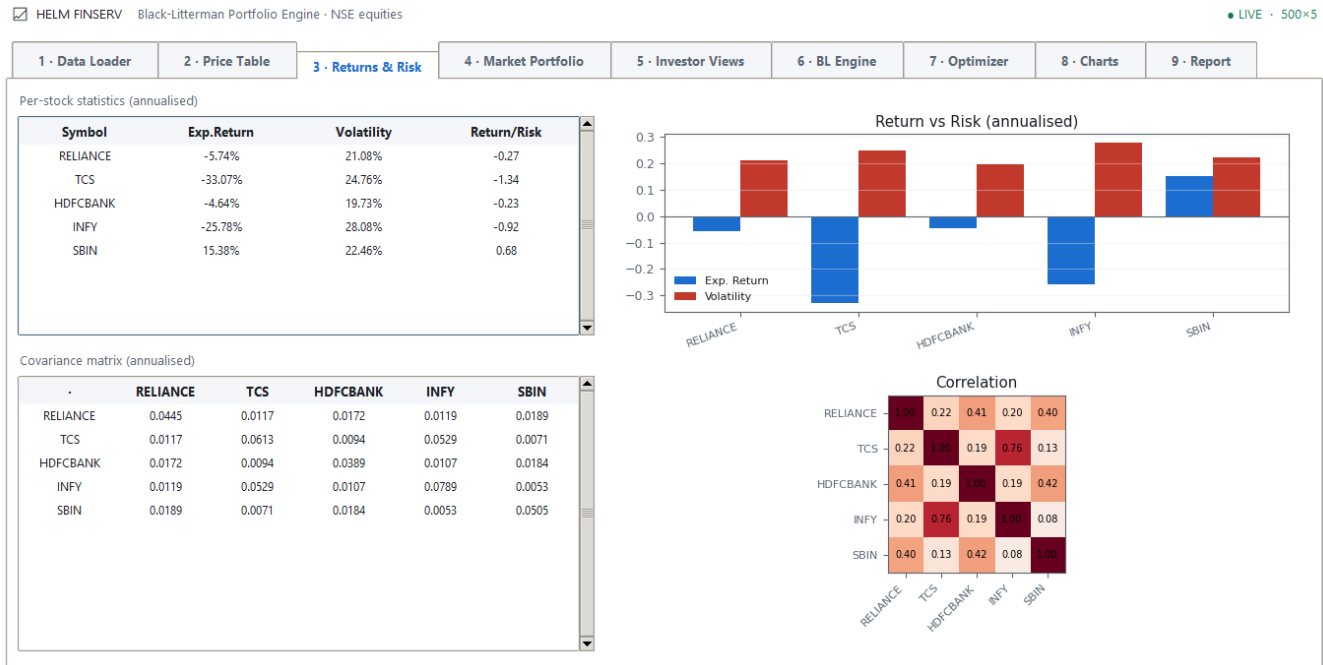
Codebase logic

```
df.iloc[:, 0] = pd.to_datetime(df.iloc[:, 0], errors="coerce")
prices = (df.dropna(subset=[df.columns[0]])
         .set_index(df.columns[0]).sort_index()
         .apply(pd.to_numeric, errors="coerce")
         .dropna(how="any"))
```

The loader parses dates, sorts observations chronologically, converts price fields to numbers and removes incomplete rows. A minimum-history validation prevents the model from running on an unusably small sample.

Return and risk estimation

Daily percentage returns are converted into annualised expected returns and a covariance matrix. The correlation heatmap highlights diversification relationships, while volatility shows the size of each asset's historical fluctuations.



Annualised return, volatility, covariance and correlation analysis

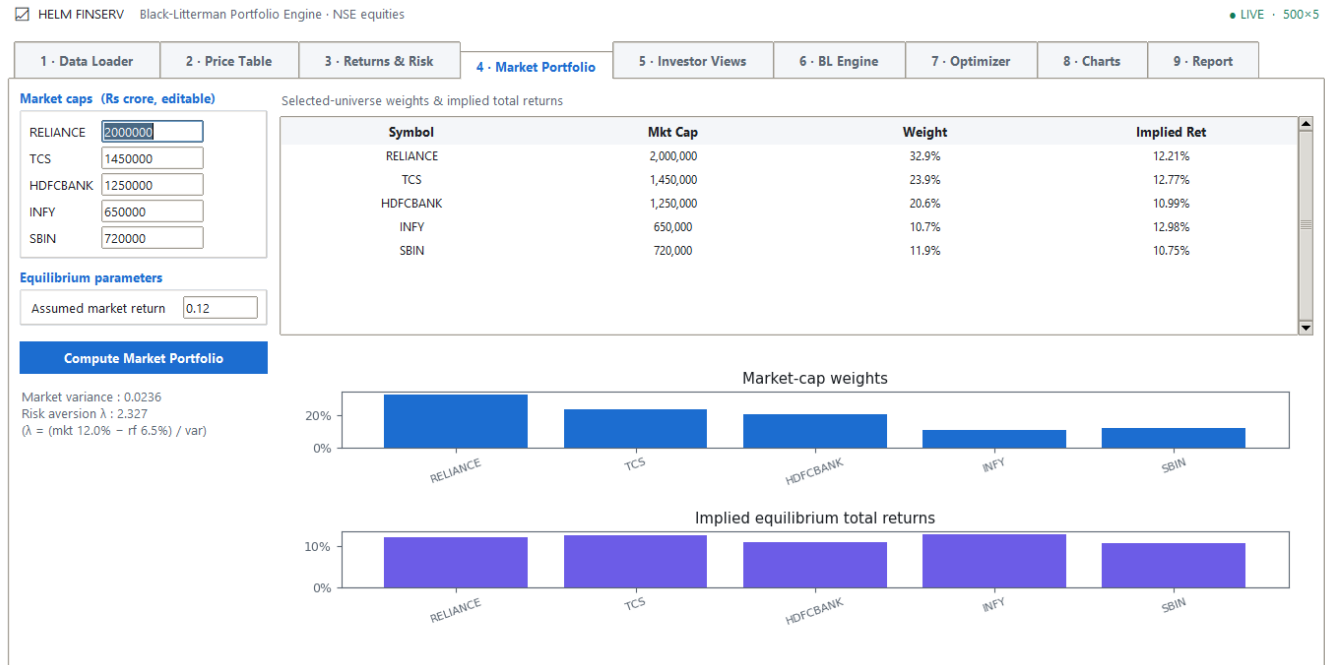
Codebase logic

```
returns = prices.pct_change().dropna(how="any")
annual_return = returns.mean() * 252
annual_covariance = returns.cov() * 252
annual_volatility = np.sqrt(np.diag(annual_covariance))
```

The covariance matrix is more important than individual volatility alone: portfolio risk depends on how assets move together, not simply on how risky each security appears in isolation.

Market portfolio and equilibrium prior

Market capitalisations are transformed into selected-universe weights. Reverse optimisation then estimates the excess returns that would make those weights consistent with the observed covariance structure and an assumed market return.



Market-cap weights and reverse-optimised equilibrium returns

Codebase logic

```
w_market = market_caps / market_caps.sum()
market_variance = w_market @ covariance @ w_market
risk_aversion = (market_return - risk_free_rate) / market_variance
prior_excess = risk_aversion * covariance @ w_market
```

This prior acts as the model's neutral starting point. It is internally consistent with market weights and risk rather than being built directly from unstable historical averages.

Investor views and confidence

The interface supports absolute views about a security's expected total return and relative views about one security outperforming another. Confidence determines how strongly each view pulls the posterior away from equilibrium.

☒ HELM FINSERV
 Black-Litterman Portfolio Engine · NSE equities
 ● LIVE · 500×5

1 · Data Loader
2 · Price Table
3 · Returns & Risk
4 · Market Portfolio
5 · Investor Views
6 · BL Engine
7 · Optimizer
8 · Charts
9 · Report

Add a view

Type
relative
Asset A
HDFCBANK
Asset B
TCS
Value / outperf. %
4
Confidence %
60
Add View

Prior uncertainty
tau (t)
0.05
Remove Selected
Clear All Views

Current views

#	Type	View	Value	Confidence
1	absolute	SBIN = 14.00%	+14.00%	65%
2	absolute	RELIANCE = 13.00%	+13.00%	55%
3	relative	HDFCBANK – TCS	+4.00%	60%

Absolute views are total returns; relative views are return spreads.
Higher confidence pulls the posterior closer to the view.

Absolute and relative investor views with explicit confidence levels

Codebase logic

```
# Absolute view: security total return
P[row, asset] = 1.0
Q[row] = stated_total_return - risk_free_rate

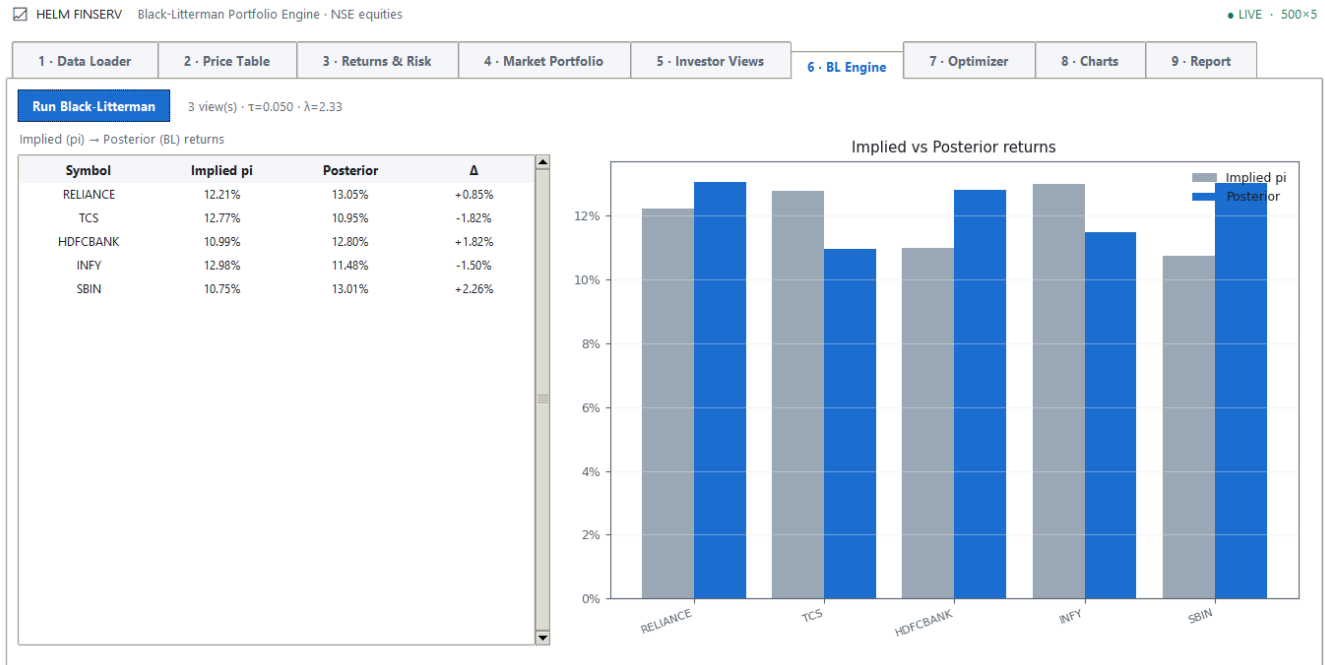
# Relative view: asset A outperforms asset B
P[row, asset_a], P[row, asset_b] = 1.0, -1.0
Q[row] = expected_outperformance
```

Absolute forecasts are converted from total return to excess return before entering the model. Relative views need no risk-free adjustment because the common risk-free component cancels.

ASHISH KUMAR A | BLACK LITTERMAN PORTFOLIO ENGINE | 6

Black Litterman posterior returns

The model combines the equilibrium prior with the view matrix, view targets and confidence-derived uncertainty. The result is a new set of posterior returns rather than an all-or-nothing replacement of the prior.



Comparison of equilibrium and Black Litterman posterior returns

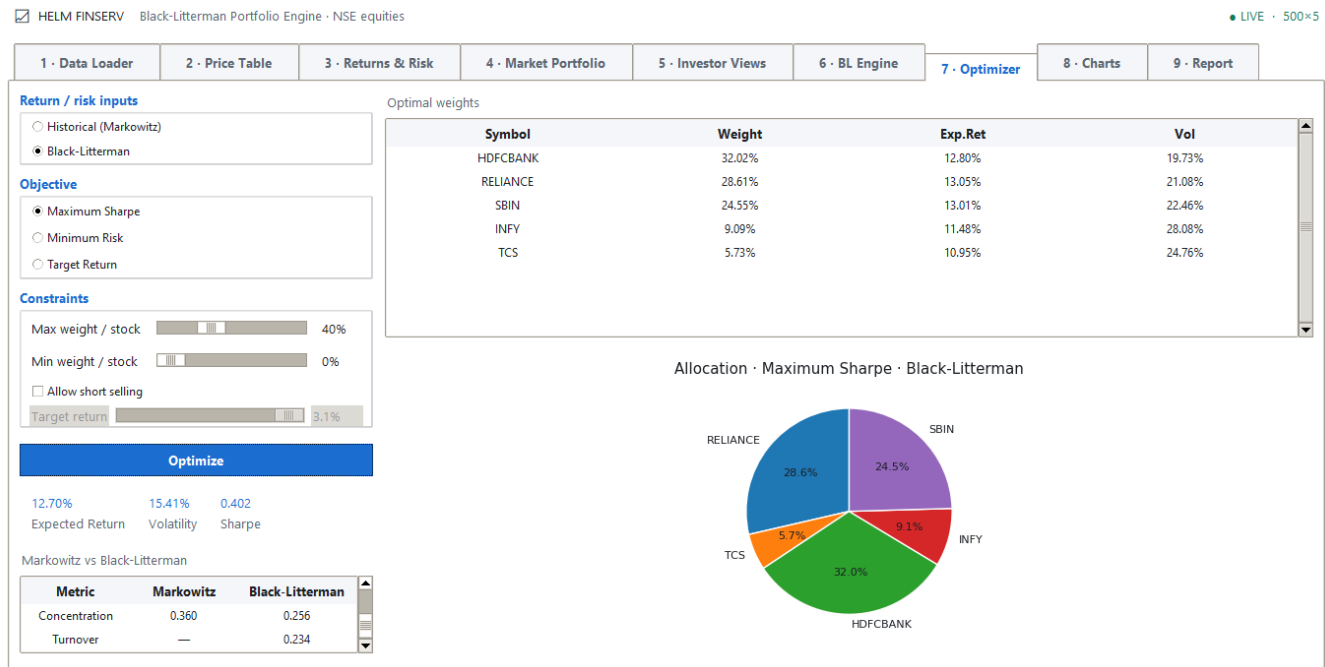
Codebase logic

```
middle = np.linalg.pinv(P @ (tau * sigma) @ P.T + omega)
posterior_excess = prior_excess + (
    tau * sigma @ P.T @ middle @ (Q - P @ prior_excess)
)
posterior_total = posterior_excess + risk_free_rate
```

Higher-confidence views receive lower uncertainty and therefore more influence. The pseudo-inverse improves numerical resilience when the view system is close to singular.

Constrained portfolio optimisation

The optimiser can search for maximum Sharpe, minimum variance or a target-return portfolio. The illustrated run uses the Black Litterman posterior and a 40 percent maximum position size to avoid a single-stock solution.



Constrained maximum-Sharpe allocation from posterior estimates

Codebase logic

```
def objective(w):
    expected = w @ mu
    volatility = np.sqrt(w @ sigma @ w)
    return -(expected - risk_free_rate) / volatility

result = minimize(objective, start, method="SLSQP",
                  bounds=weight_bounds,
                  constraints={"type": "eq", "fun": lambda w: w.sum() - 1})
```

Multiple feasible starting portfolios are tested. This reduces dependence on a single starting guess, while the full-investment equation keeps weights summing to 100 percent.

Efficient frontier comparison

The frontier maps the best attainable expected return for different levels of estimated risk. Plotting historical and posterior frontiers together makes the effect of the Black Litterman update visible.



Efficient-frontier comparison and feasible portfolio cloud

Codebase logic

```
minimum_risk = optimise(mu, sigma, objective="min_variance")
for target in np.linspace(minimum_risk.expected_return, max_return, 30):
    portfolio = optimise(mu, sigma, objective="target",
                        target_return=target)
    frontier.append((portfolio.volatility,
                    portfolio.expected_return,
                    portfolio.sharpe))
```

Every point is generated under the same weight restrictions as the final optimiser, keeping the frontier and final allocation methodologically consistent.

Reporting and decision output

The final report consolidates assumptions, market-implied returns, views, posterior estimates and portfolio weights. It creates an auditable chain from input assumptions to the final allocation.

HELM FINSERV Black-Litterman Portfolio Engine · NSE equities ● LIVE · 500×5

1 · Data Loader 2 · Price Table 3 · Returns & Risk 4 · Market Portfolio 5 · Investor Views 6 · BL Engine 7 · Optimizer 8 · Charts 9 · Report

Export Weights CSV Save Report (.txt)

```

BLACK-LITTERMAN REPORT
=====
Data: live (500 sessions) Risk-free: 6.50% lambda: 2.33 tau: 0.050

Selected-Universe Market-Implied Total Returns:
RELIANCE 12.21%
TCS 12.77%
HDFCBANK 10.99%
INFY 12.98%
SBIN 10.75%

Investor Views:
SBIN = 14.00% (confidence 65%)
RELIANCE = 13.00% (confidence 55%)
HDFCBANK - TCS = +4.00% (confidence 60%)

Posterior Total Returns (BL):
RELIANCE 13.05%
TCS 10.95%
HDFCBANK 12.80%
INFY 11.48%
SBIN 13.01%

FINAL PORTFOLIO — Maximum Sharpe · Black-Litterman
-----
HDFCBANK 32.02%
RELIANCE 28.61%
SBIN 24.55%
INFY 9.09%
  
```

Final model report containing assumptions, views, posterior returns and weights

Codebase logic

```

comparison = pd.DataFrame({
    "Historical": [hist_return, hist_vol, hist_sharpe, hist_hhi],
    "Black-Litterman": [bl_return, bl_vol, bl_sharpe, bl_hhi],
}, index=["Expected return", "Volatility", "Sharpe", "Concentration"])
comparison.loc["Turnover", "Black-Litterman"] = turnover(hist_w, bl_w)
  
```

Return and volatility describe model expectations, Sharpe measures excess return per unit of risk, concentration captures allocation dependence, and turnover quantifies how much the portfolio changes between methods.

What the project achieved

Capability	Achievement
Equilibrium prior	Recovered return expectations implied by market-cap weights and covariance.
Structured views	Encoded both absolute and relative forecasts in a common mathematical form.
Confidence control	Translated view confidence into uncertainty rather than treating every opinion equally.
Posterior estimates	Blended market information and investment judgement through Bayesian updating.
Practical allocation	Applied full-investment and position-size constraints to the optimiser.
Model comparison	Evaluated return, volatility, Sharpe, concentration and turnover side by side.

Why this matters

The project demonstrates more than formula implementation. It connects data preparation, portfolio theory, numerical optimisation and model governance in one explainable workflow. The most important design choice is the separation of the market prior, investor views and confidence: each component can be inspected, challenged and changed independently.

Limitations

- The five-stock portfolio is a selected universe, not a proxy for the entire equity market.
- Market capitalisations, expected market return and views are modelling assumptions.
- Historical covariance may change, especially during stressed markets.
- The current study excludes taxes, transaction costs, liquidity and market impact.
- Modelled results are in-sample estimates and do not establish future profitability.

Recommended next extensions

Walk-forward testing, shrinkage covariance, sector limits, turnover penalties, transaction costs, drawdown analytics and benchmark attribution would move the engine closer to production research.

LinkedIn project summary

Built a Python-based Black Litterman portfolio optimisation engine that starts with market-implied equilibrium returns, incorporates confidence-weighted absolute and relative investment views, and compares the resulting allocation with a historical Markowitz portfolio. The workflow covers return and covariance estimation, reverse optimisation, Bayesian updating, constrained maximum-Sharpe optimisation, efficient-frontier analysis, concentration and turnover measurement, and automated reporting.

Technology and quantitative methods

Area	Tools and methods
Language	Python
Data	Pandas and NumPy
Optimisation	SciPy SLSQP and linear programming
Portfolio theory	Markowitz, reverse optimisation and Black Litterman
Risk analytics	Annualised covariance, volatility, Sharpe and concentration HHI
Validation	Input checks, feasible-bound checks and multi-start optimisation

Repository structure

```
black-litterman-research-case-study/  
├─ black_litterman_engine.py # quantitative engine  
├─ README.md                # methodology and results  
├─ requirements.txt         # Python dependencies  
├─ sample_data/             # research inputs  
└─ screenshots/             # application evidence
```

Run locally

```
pip install -r requirements.txt  
python black_litterman_engine.py  
python black_litterman_engine.py --prices your_prices.csv
```

Educational research project. The outputs are model estimates, not investment advice or a recommendation to transact.